Deep Learning and Computation

Images: convolutional networks and transfer learning — Lecture 23 (Chapter 10, Part 4)

Amir Farbin

Part 4 — Images: convolutional networks and transfer learning

The problem

- Separate cats from dogs, "but for some reason I only have 100 pictures of cats and 100 pictures of dogs"
- That is a normal size for a real problem: a lab with a hundred patients' X-rays, labelled by a radiologist
- Two problems at once: **very little data**, and **little computing**
- Today: first a network trained **from scratch**, which fails; then **transfer learning**, which works

An image is a tensor

the photo



red channel



green channel



blue channel



5×5 red values

103	107	164	135	92
100	156	139	112	139
144	116	83	117	147
136	82	93	123	86
120	95	129	139	92

one image: 224 × 224 × 3 numbers from 0 to 255. A batch of 10: shape (10, 224, 224, 3)

- Height × width × 3 **channels** (red, green, blue), each **pixel** value 0–255
- A batch adds a first index: 4 dimensions. Networks need equal sizes, so every image is **resized** (224 × 224 here)

The folders are the labels

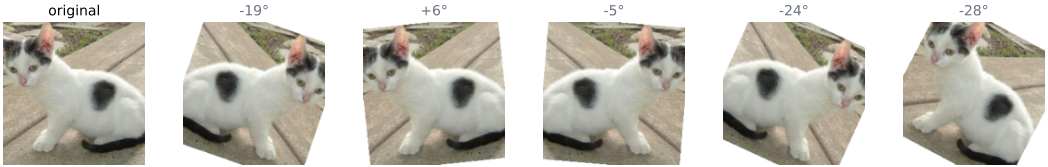
```
train_ds, val_ds = keras.utils.image_dataset_from_directory(
    "PetImages", validation_split=0.2, subset="both", seed=1337,
    image_size=(224, 224), batch_size=10)
```

- One sub-folder per class: `PetImages/Cat/`, `PetImages/Dog/`. Labels are folder names, in alphabetical order: Cat = 0, Dog = 1
- 20% held out for **validation**. The **seed** fixes which files: same seed, same split, same crash to chase
- The datasets are **generators**: batches are read, decoded and resized as they are needed

Bad files, and the cost of reading

- Downloads contain broken images. "You're trained for a long time, and suddenly you pick up an image that's not good, and that makes the whole job crash." Check them first; skip and list the bad ones
- The notebook keeps files that say `JFIF` in their first bytes (a valid JPEG header), and only ~100 per class
- **Decoding** compressed images takes time; at scale it dominates. CPUs prepare the next batch while the GPU trains (`prefetch`): GPUs that cost thousands shouldn't sit idle

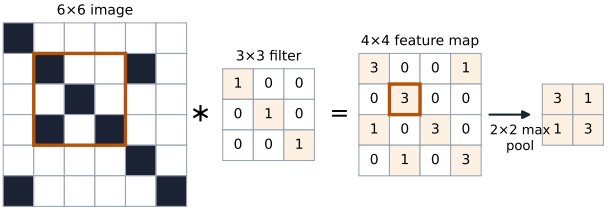
Data augmentation



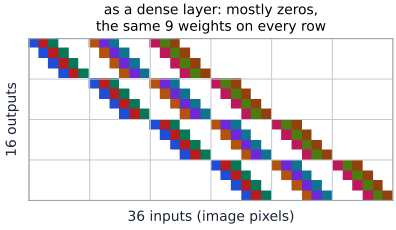
still a cat: every epoch sees a different version of each training image

- **Augmentation:** random changes that keep the label: flip, rotate (here up to $\pm 36°$), zoom
- Two purposes: **more training examples** from the same photos, and **invariance**: the answer shouldn't change when the cat is turned or moved to the corner
- Only during training. Augmentation layers are functions, so they can live inside the model

Convolutional networks: a dense layer with shared weights



highlighted window: sum of image × filter = 3 (the diagonal it matches)



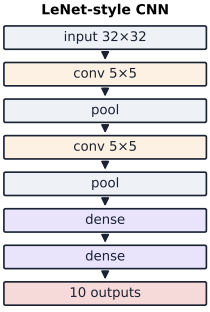
colour = which of the 9 filter weights

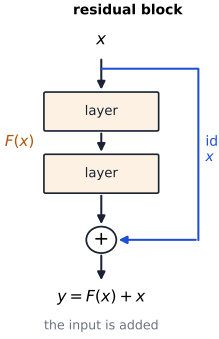
- A small filter slides over the image → feature map; pooling shrinks; stack
- As a dense matrix: **mostly zeros**, and the same few weights repeated

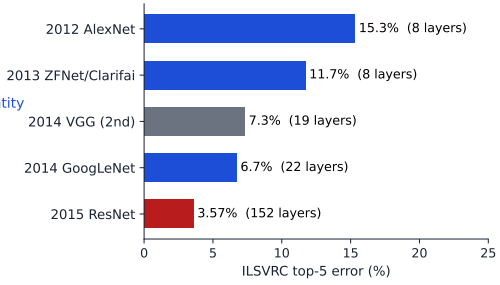
A small CNN from scratch

- `Rescaling(1/255)`: pixels to 0–1. `Conv2D(128, 3, strides=2, padding="same")`: 128 filters of 3×3, step 2, edges padded
- **Batch normalization:** rescales a layer's outputs over the batch; helps the descent. Keeps running averages for prediction
- **Separable convolutions**, **max pooling**, and a **residual** (skip) connection per block; then **global average pooling**: one number per feature map
- One output, a **logit** (the value before the sigmoid). `from_logits=True` lets the loss apply the sigmoid: numerically stabler

Architectures, 1998–2015 and after

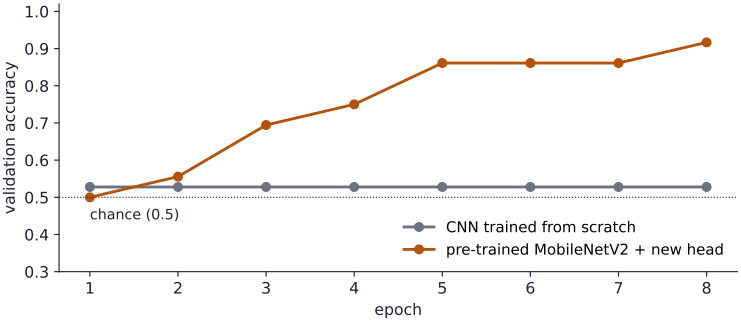






- LeNet (1989–98), AlexNet (2012, two gaming GPUs), VGG and GoogLeNet (2014), ResNet (2015, 152 layers)
- Residual block: $y = F(x) + x$. The same skip connection is in every transformer

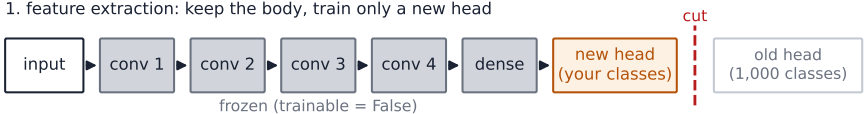
From scratch: it doesn't learn



- From scratch: training accuracy creeps up, but validation stays at 0.53, the same answer for every image
- 148 photos are far too few to learn what edges, fur and eyes look like. Transfer: 0.92 by epoch 8, at a seventh of the time per epoch

Transfer learning

1. feature extraction: keep the body, train only a new head

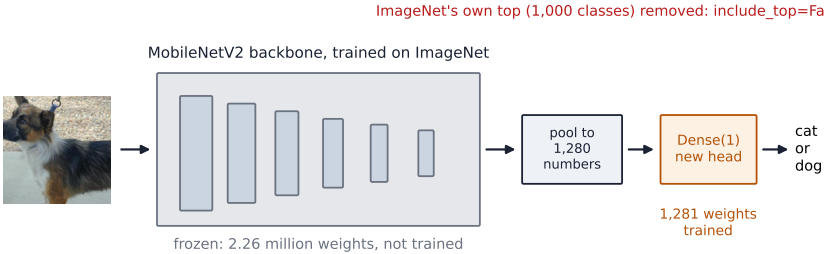


2. optional fine-tuning: then unfreeze and train everything gently



- What a network learns belongs to the **domain**, not just the task
- Download a **pre-trained** network (trained by someone else on a huge dataset), replace its last layer, train that; then perhaps **fine-tune** everything

The transfer model



- `MobileNetV2(weights="imagenet", include_top=False)`: a small CNN for phones, trained on ImageNet's 1,000 classes, without its last layers ("they call it top; I would call it the bottom")
- **Freeze** it (`backbone.trainable = False`) and train only the new head: 1,281 weights
- The backbone expects its own input scaling: `preprocess_input`

Fine-tuning, and the moral

- **Fine-tuning:** unfreeze the backbone and train everything a little, so its features suit your images. Worth it when your images are unlike ImageNet's; it costs much more computing
- Keras' recipe: call the backbone with `training=False`, set `trainable = True`, **recompile**, and use a tiny learning rate (1e-5)
- Little data, or little computing: **transfer learning**. For images, audio and text alike; today's language models are used the same way
- Start from the Keras examples: "whatever you want to do, you will find an example that will get you quite a ways there"

Vocabulary

- **pixel, channel, image tensor, batch, resize, JPEG / JFIF, corrupted file**
- **class-per-folder layout,** `image_dataset_from_directory`**, validation split, seed, generator, prefetch**
- **data augmentation, invariance,** `RandomFlip`**,** `RandomRotation`
- **convolution, filter (kernel), stride, padding, feature map, pooling (max, global average)**
- **batch normalization, separable convolution, residual (skip) connection, logit,** `from_logits`
- **pre-trained model, ImageNet, backbone, top / head,** `include_top`**, freezing,** `preprocess_input`
- **transfer learning, fine-tuning, inference mode (**`training=False`**)**

What comes next

- Lecture 24: **parallel computing**: processes, threads and queues, which is how data loaders keep the GPU fed
- Your Kaggle project: datasets are chosen; next, the first look at your data